

🔍 কোড ওয়াকথ্রু গাইড — "স্যার, কোথায় কানেক্ট হয়েছে, দেখাই"

স্যার যদি বলেন — "দেখান, আপনার কোডে ফ্রন্টএন্ড থেকে ব্যাকএন্ডে কানেকশন কোথায়?" — এই গাইড পড়ে আপনি হুবহু ফাইল ও লাইন নম্বর দেখিয়ে ব্যাখ্যা করতে পারবেন। নিয়ম আগের মতো: টেকনিক্যাল টার্ম ইংরেজিতে, ব্যাখ্যা সহজ বাংলায়।

১. VS Code-এ প্রজেক্ট খুললে আমার কোড কোথায় পাবেন

স্যার প্রজেক্ট ফোল্ডার খুললে **Explorer** প্যানেলে এই পাথগুলোতে আমার কাজ আছে:

```
shurokkha/
├─ apps/admin/src/components/app/
│   └─ operations-workspace.tsx      ← ফ্রন্টএন্ড UI (ট্যাব, ফর্ম, টেবিল, হ্যান্ডলার)
├─ apps/admin/src/lib/
│   └─ api.ts                        ← ব্যাকএন্ডের ঠিকানা (base URL) ঠিক করা
├─ packages/contracts/src/
│   └─ index.ts                      ← TypeScript টাইপ (ডাটার কাঠামোর চুক্তি)
├─ packages/api-client/src/
│   └─ index.ts                      ← API কল করার SDK (fetch র‍্যাপার)
├─ services/api/
│   └─ routes/api.php                ← কোন URL কোন কন্ট্রোলারে যাবে
│       └─ app/Http/Controllers/Api/V1/Admin/
│           ├── ShelterController.php ← শেল্টারের রিকোয়েস্ট হ্যান্ডলার
│           ├── WarehouseController.php ← গুদামের রিকোয়েস্ট হ্যান্ডলার
│           └─ DonationController.php ← দানের রিকোয়েস্ট হ্যান্ডলার
│       └─ app/Models/
│           ├── Shelter.php           ← shelters টেবিলের মডেল
│           ├── Warehouse.php         ← warehouses টেবিলের মডেল
│           └─ Donation.php           ← donations টেবিলের মডেল
└─ database/
    ├── migrations/08_create_shelters_table.sql
    ├── migrations/09_create_warehouses_table.sql
    ├── migrations/10_create_donations_table.sql
    ├── admin_shelter_capacity_report.sql ← JOIN/সাবকোয়ারি ডেমো
    └─ seed_relief_operations.sql      ← সিড ডাটা
```

💡 VS Code টিপস (স্যারকে দেখানোর সময়):

- **Ctrl + P** চেপে ফাইলের নাম লিখলে সরাসরি ফাইল খোলে।
- **Ctrl + G** চেপে লাইন নম্বর লিখলে সরাসরি সেই লাইনে যায়।

- এই গাইডের প্রতিটি রেফারেন্স ফাইল:লাইন আকারে দেওয়া — যেমন `operations-workspace.tsx:188`।

২. ডাটা ফ্লো #১ — লিস্ট দেখানো (READ)

দৃশ্য: পেজ খোলার সাথে সাথে Shelters ট্যাবে ডাটাবেজের ১০টি শেল্টার দেখা যায়। ডাটা কোন পথে এলো?

ধাপ ১ — পেজ লোড হলে ফেচ শুরু (`operations-workspace.tsx:360`)

```
360:   fetchBackendData()  
361: }, [fetchBackendData])
```

→ কম্পোনেন্ট মাউন্ট হওয়ার সাথে সাথে (ইউজার পেজ খুললেই) `fetchBackendData` ফাংশনটি চলে। এটি `React.useEffect` -এর ভেতরে, তাই একবারই স্বয়ংক্রিয়ভাবে চলে।

ধাপ ২ — `api` অবজেক্ট তৈরি (`operations-workspace.tsx:171`)

```
171: const api = React.useMemo(() => getShurokkhaApi(), [])
```

→ `getShurokkhaApi` আসে `apps/admin/src/lib/api.ts:44` থেকে। ওখানেই ব্যাকএন্ডের ঠিকানা নির্ধারণ হয়:

```
// apps/admin/src/lib/api.ts  
3:  const DEFAULT_API_ORIGIN = "http://localhost:8000"    // ব্যাকএন্ড কোথায় চলে  
44: export function getShurokkhaApi() {  
46:   baseUrl: getApiBaseUrl(),                            // ওই ঠিকানা ব্যবহার হয়
```

🐞 স্যারকে বলুন: "স্যার, ফ্রন্টএন্ড `lib/api.ts` -এর ৩ নম্বর লাইন থেকে ব্যাকএন্ডের base URL নেয় — `http://localhost:8000`। এটাই কানেকশনের শুরু।"

ধাপ ৩ — সাতটি API কল একসাথে (`operations-workspace.tsx:183-191`)

```
183:   ] = await Promise.all([  
188:     api.admin.shelters.list(),      // ← আমার যোগ করা কল  
189:     api.admin.warehouses.list(),    // ← আমার যোগ করা কল  
190:     api.admin.donations.list(),     // ← আমার যোগ করা কল  
191:   ])
```

→ `Promise.all` দিয়ে একসাথে সাতটি রিকোয়েস্ট পাঠানো হয় — দ্রুততার জন্য। এর মধ্যে ১৮৮-১৯০ আমার তিন মডিউলের কল।

ধাপ ৪ — SDK আসল HTTP রিকোয়েস্ট বানায় (packages/api-client/src/index.ts:497-502)

```
497:     shelters: {
499:       client.get<ApiResponse<ShelterRecord[]>>("/v1/admin/shelters"),
501:       client.post<ApiResponse<ShelterRecord>>("/v1/admin/shelters", input),
502:       remove: (id: number) => client.delete<void>(`/v1/admin/shelters/${id}`),
```

→ `list()` মানে `client.get("/v1/admin/shelters")` — অর্থাৎ পুরো URL হয়

`http://localhost:8000/api/v1/admin/shelters` ।

কানেকশনের দুটি গুরুত্বপূর্ণ লাইন:

```
87:     this.credentials = options.credentials ?? "include" // কুকি পাঠানো হয়
212:     if (xsrftoken && !headers["X-XSRF-TOKEN"]) {
213:       headers["X-XSRF-TOKEN"] = xsrftoken // CSRF নিরাপত্তা টোকেন
```

→ `credentials: "include"` মানে ব্রাউজারের কুকি (সেশন) রিকোয়েস্টের সাথে যায়; আর `X-XSRF-TOKEN` হেডার দিয়ে লারাভেল যাচাই করে রিকোয়েস্টটি আসল ব্রাউজার থেকে এসেছে।

ধাপ ৫ — রুট ঠিক করে দেয় কোন কন্ট্রোলার (services/api/routes/api.php:63)

```
63: Route::get('/shelters', [ShelterController::class, 'index'])->name('shelters.index');
```

→ GET `/api/v1/admin/shelters` এলে লারাভেল `ShelterController` -এর `index` মেথড চালাবে। এটাই ফ্রন্টএন্ড → ব্যাকএন্ড কানেকশনের সেতু।

ধাপ ৬ — কন্ট্রোলার ডাটাবেজ থেকে পড়ে (ShelterController.php:12-15)

```
12:     public function index(): JsonResponse
14:     {
15:         $shelters = Shelter::latest()->get(); // Eloquent → SQL কোয়ারি
        return response()->json(['data' => $shelters]);
```

→ `Shelter::latest()->get()` লারাভেল নিজে SQL বানায়: `SELECT * FROM shelters ORDER BY created_at DESC` — এবং **MySQL-এর shelters টেবিল** থেকে ডাটা আনে।

ধাপ ৭ — ডাটা ফ্রন্টএন্ডে ফিরে টেবিলে বসে (operations-workspace.tsx:206)

```
206:     setShelters(sheltersRes.data) // React state-এ ডাটা
209:     setIsOffline(false) // মানে: ব্যাকএন্ডের সাথে কানেক্টেড!
```

→ এরপর `shelters.map(...)` দিয়ে প্রতিটি রো HTML টেবিলের সারি হয়।

 পুরো READ ফ্লো এক নিঃশ্বাসে: "স্যার, পেজ খুললে ৩৬ লাইনে `fetchBackendData` চলে → ১৮ লাইনে `shelters.list()` কল হয় → `api-client`-এর ৪৯৯ লাইনে GET রিকোয়েস্ট বানে → `api.php` -র ৬৩ লাইনের

রুট ধরে `ShelterController`-এর `index`-এ যায় → ১৪ লাইনে Eloquent দিয়ে MySQL-এর `shelters` টেবিল থেকে পড়ে → JSON হয়ে ফিরে ২০৬ লাইনে `state`-এ বসে → টেবিলে দেখা যায়।"

৩. ডাটা ফ্লো #২ — ফর্ম সাবমিট (WRITE)

দৃশ্য: "Register Shelter" ফর্মে নাম, capacity, occupancy, area দিয়ে Save চাপলে ডাটাবেজে নতুন রো যুক্ত হয়।

ধাপ ১ — সাবমিট হ্যান্ডলার (`operations-workspace.tsx:574-586`)

```
575:   if (!shelterForm.shelter_name || !shelterForm.capacity) { // ফ্রন্টএন্ড যাচাই
580:     const payload = {
581:       shelter_name: shelterForm.shelter_name,
582:       area_id: Number(shelterForm.area_id),
583:       capacity: Number(shelterForm.capacity),
584:       occupancy: Number(shelterForm.occupancy || 0),
585:       status: shelterForm.status,
586:     }
```

→ লক্ষ্য করুন: **payload**-এর কি-গুলো হুবহু ডাটাবেজের কলামের নাম (`shelter_name`, `area_id`, `capacity`, `occupancy`, `status`)। এটাই ফ্রন্টএন্ড-ডাটাবেজ মিলের প্রথম প্রমাণ।

ধাপ ২ — POST কল (`operations-workspace.tsx:590-592`)

```
590:     const res = await api.admin.shelters.create(payload) // POST পাঠায়
591:     setShelters([res.data, ...shelters]) // নতুন রো তালিকায়
592:     toast.success(`Shelter saved to backend (ID: ${res.data.shelter_id})`)
```

ধাপ ৩ — SDK → HTTP POST (`packages/api-client/src/index.ts:501`)

```
501:     client.post<ApiResource<ShelterRecord>>("/v1/admin/shelters", input),
```

ধাপ ৪ — রুট (`services/api/routes/api.php:64`)

```
64: Route::post('/shelters', [ShelterController::class, 'store']->name('shelters.store'));
```

ধাপ ৫ — কন্ট্রোলারে ভ্যালিডেশন + INSERT (ShelterController.php:20-29)

```
20:     $validated = $request->validate([
21:         'shelter_name' => 'required|string|max:150',
22:         'capacity' => 'required|integer|min:0',
23:         'occupancy' => 'required|integer|min:0',
24:         'area_id' => 'nullable|integer',
25:         'status' => 'sometimes|string|max:50',
26:     ]);
28:     $shelter = Shelter::create($validated);    // INSERT INTO shelters (...)
29:     return response()->json(['data' => $shelter], 201);
```

→ `Shelter::create()` লারাভেল SQL বানায়: `INSERT INTO shelters (shelter_name, capacity, occupancy, area_id, status) VALUES (...)` — ডাটা **MySQL-এ স্থায়ীভাবে** জমা হয়। `201` স্ট্যাটাস মানে "তৈরি হয়েছে"।

🐞 **WRITE ফ্লো এক নিঃশ্বাসে:** "স্যার, ৫৮০ লাইনে `payload` বানে যেখানে কি-গুলো ডাটাবেজের কলামের নামের হুবহু সমান → ৫৯০ লাইনে `POST` → `api-client` ৫০১ → রুট ৬৪ → `store` মেথডে ২০-২৬ লাইনে ভ্যালিডেশন → ২৮ লাইনে `create` মানে `INSERT` → ২০১ রেসপন্স → ৫৯১ লাইনে নতুন রো তালিকায়।"

DELETE ফ্লো (এক লাইনে)

`operations-workspace.tsx:620` (`remove`) → `api-client:502` (`DELETE`) → `api.php:65` → `ShelterController.php:32-35` (`destroy`) → `DELETE FROM shelters WHERE shelter_id = ?` → `204` রেসপন্স।

৪. ফ্রন্টএন্ড কোথায় ডাটাবেজের সাথে মিলেছে? (ম্যাপিং টেবিল)

স্যার জিজ্ঞেস করলে — "ফ্রন্টএন্ডের কোড আর ডাটাবেজের কলাম মিলবে কীভাবে?" — এই টেবিল দেখান:

ফর্ম ফিল্ড (UI)	payload কি (:580-586)	ব্যাকএন্ড ভ্যালিডেশন (:20-26)	DB কলাম (08_...sql)	TS টাইপ (contracts:203-213)
Shelter Name	shelter_name	required string max:150	shelter_name VARCHAR(150)	shelter_name: string
Affected Area	area_id	nullable integer	area_id INT (FK)	area_id: number null
Capacity	capacity	required integer min:0	capacity INT UNSIGNED	capacity: number
Occupancy	occupancy	required integer min:0	occupancy INT UNSIGNED	occupancy: number
Status	status	sometimes string max:50	status VARCHAR(50)	status: "open" "full" "closed"

মিলের চারটি স্তর (স্যারকে বোঝান):

1. **নামের মিল:** payload-এর কি = ডাটাবেজের কলামের নাম (হুবহু)।
2. **টাইপের মিল:** contracts -এর TypeScript ইন্টারফেস (ShelterRecord , contracts:203) ডাটাবেজের কলামের টাইপের প্রতিচ্ছবি — number ↔ INT , string ↔ VARCHAR ।
3. **নিয়মের মিল:** ভ্যালিডেশন রুল ডাটাবেজের সীমার সমান — যেমন max:150 কারণ কলাম VARCHAR(150); min:0 কারণ INT UNSIGNED ।
4. **সম্পর্কের মিল:** area_id ফরেন-কি — মাইগ্রেশনের ১৯ লাইনে (08_create_shelters_table.sql:19) fk_shelters_area কনস্ট্রেন্ট, আর মডেলে belongsTo (Shelter.php:23-26)।

🐞 **উত্তর:** "স্যার, মিলটা চার স্তরে রাখা: নাম এক, টাইপ এক, সীমা এক, আর সম্পর্ক এক। তাই ফ্রন্টএন্ড থেকে যা আসে, ডাটাবেজ হুবহু তা-ই নেয় — কোনো মিসম্যাচ হয় না।"

৫. স্যারকে VS Code-এ লাইভ দেখানোর স্ক্রিপ্ট

স্যারকে বলুন: "স্যার, আমি ধাপে ধাপে দেখাচ্ছি —"

1. Ctrl+P → operations-workspace.tsx → Ctrl+G → 171 → "এখানে api অবজেক্ট — এটাই ফ্রন্টএন্ডের দরজা।"
2. Ctrl+G → 188 → "এই তিন লাইন আমার যোগ করা API কল — শেল্টার, গুদাম, দান।"
3. Ctrl+P → api-client/src/index.ts → Ctrl+G → 499 → "এখানে কলটা আসল HTTP রিকোয়েস্টে বদলায় — GET /v1/admin/shelters ।"
4. Ctrl+P → api.php → Ctrl+G → 63 → "ব্যাকএন্ডে এই রুট রিকোয়েস্ট ধরে কন্ট্রোলারে পাঠায় — এটাই কানেকশনের সেতু।"
5. Ctrl+P → ShelterController.php → Ctrl+G → 14 → "এই এক লাইন ডাটাবেজ থেকে পড়ে — Eloquent ORM নিজে SQL বানায়।"

6. `Ctrl+P` → `Shelter.php` → `Ctrl+G` → `13` → "মডেল বলে দেয় টেবিলের আইডি কোনটা, আর ১৫-২১ লাইনে কোন কলামে লেখা যাবে।"
7. `Ctrl+P` → `08_create_shelters_table.sql` → `Ctrl+G` → `19` → "আর সবশেষে ডাটাবেজে ফরেন-কি — এটাই টেবিলগুলোর সম্পর্ক।"

🐞 **শেষে বলুন:** "স্যার, অর্থাৎ ফ্রন্টএন্ডের ১৮ লাইন থেকে ডাটাবেজের ১৯ লাইন পর্যন্ত — পুরো পথ আমি দেখাতে পারি। প্রতিটি ধাপ আলাদা ফাইলে, কিন্তু নাম ও টাইপের মিলের কারণে পুরো চেইন এক সূতোয় বাঁধা।"

৬. কানেকশন নিয়ে সম্ভাব্য প্রশ্নোত্তর

প্রশ্ন	উত্তর (ফাইল:লাইন)
ফ্রন্টএন্ড কীভাবে জানে ব্যাকএন্ড কোথায়?	<code>lib/api.ts:3</code> — <code>DEFAULT_API_ORIGIN = "http://localhost:8000"</code>
রিকোয়েস্টে কুকি/সেশন কীভাবে যায়?	<code>api-client:87</code> — <code>credentials: "include"</code>
CSRF নিরাপত্তা কোথায়?	<code>api-client:212-213</code> — <code>X-XSRF-TOKEN</code> হেডার
কোন URL কোন কন্ট্রোলারে যাবে কে ঠিক করে?	<code>routes/api.php:63-73</code> — রুট ডেফিনিশন
লারাভেল কীভাবে টেবিলের নাম জানল?	মডেলের ক্লাস-নাম থেকে নিয়ম: <code>Shelter</code> → <code>shelters</code> টেবিল
আইডি কলাম <code>id</code> না হলে?	<code>Shelter.php:13</code> — <code>\$primaryKey = 'shelter_id'</code> বলে দেওয়া
ব্যাকএন্ড বন্ধ থাকলে কী হয়?	<code>operations-workspace.tsx:210-215</code> — <code>catch</code> ব্লক <code>isOffline=true</code> করে লোকাল মক ডাটা দেখায়
ফ্রন্টএন্ড-ব্যাকএন্ড আলাদা পোর্টে চলে, সমস্যা হয় না কেন?	<code>.env</code> -এ <code>FRONTEND_URLS</code> — লারাভেল CORS অনুমতি দেয়

মনে রাখুন: প্রতিটি কানেকশনের একটি সেতু-লাইন আছে — ফ্রন্টএন্ডে `188/590/620`, SDK-তে `499/501/502`, রুটে `63/64/65`, কন্ট্রোলারে `14/28/34`, মডেলে `13/15/23`। এই লাইনগুলো মনে রাখলে স্যার যেখানেই আঙুল রাখুন, আপনি পুরো পথ দেখাতে পারবেন। 🌀